HIDROSMART

Manual de instalación de HidroSmart

Preparación del ambiente local y puesta en marcha

Versión documental consolidada 1.0
Fecha de corte: 2026-10-09
Código documental: DOC-09

Ampliación del procedimiento de instalación

Esta versión desarrolla la instalación local que está descrita en PT_HS. Los pasos destructivos se separan de la instalación normal y se advierte cuándo una verificación no está cerrada. No se incluyen valores reales de contraseñas, tokens, SMTP ni Wi-Fi.

1. Alcance

Este manual cubre la instalación local de los componentes versionados en PT_HS. No describe una liberación productiva ni contiene secretos. La instalación debe realizarse en una copia de trabajo y con las variables del entorno protegidas.

2. Requisitos

Recurso	Uso
Docker y Docker Compose	Servicios de PostgreSQL, Mosquitto, backend, frontend y bootstrap.
Node/npm	Desarrollo o ejecución fuera de contenedor del backend/frontend.
PlatformIO	Compilación y carga del firmware ESP32.
ESP32 y sensor	Prueba física opcional; requiere alimentación y red.
Development build móvil	Aprovisionamiento BLE; Expo Go no es suficiente para BLE nativo.

3. Configuración segura

- Copiar los archivos de ejemplo de variables según README vigente.
- Definir valores locales para BD, JWT, MQTT y VITE_API_URL sin publicarlos.
- Usar nombres de servicio Docker dentro de la red de Compose; usar puertos publicados solo desde el host.
- No reutilizar credenciales de desarrollo en producción.
- Revisar docker-compose.yml antes de iniciar, porque puertos y argumentos de build pueden cambiar.

4. Instalación de servicios

1. Ubicarse en la raíz PT_HS.
2. Construir y levantar el conjunto definido por docker-compose.yml.
3. Esperar PostgreSQL y ejecutar el bootstrap/Liquibase del proyecto.
4. Verificar que el changelog esté actualizado y que los servicios estén saludables.
5. Abrir la interfaz en localhost:5173 y verificar API/backend en localhost:3000 según el compose.
6. Verificar broker local en localhost:1883 solo desde el ambiente permitido.

5. Instalación del firmware

7. Abrir firmware en PlatformIO y seleccionar esp32dev.
8. Compilar y revisar memoria antes de cargar.
9. Cargar firmware al ESP32 y abrir monitor a 115200.
10. Usar el flujo BLE para guardar SSID, host, puerto y deviceCode.
11. Confirmar estado MQTT y publicar telemetría controlada.
12. No cargar datos de usuarios ni credenciales reales en evidencias.

6. Verificación posterior

Verificación	Aprobación
Contenedores	Postgres, broker, backend y frontend responden.
BD	Liquibase válido y tablas disponibles.
API	Autenticación y ruta protegida responden con datos de prueba.
MQTT	Mensaje de prueba aparece en logs y/o lectura persistida.
Frontend	Login, dashboard y consulta de dispositivo funcionan.
Firmware	ESP32 publica estado y telemetría; RSSI visible.

7. Desinstalación y recuperación

Antes de eliminar volúmenes o reiniciar datos, obtener backup y confirmar el objetivo. No ejecutar acciones destructivas sobre una base compartida. El material revisado no aporta una receta validada de restauración completa; DOC-11 mantiene esta actividad como pendiente prioritaria.

Nota de control documental. Puertos y servicios derivados de docker-compose.yml y documentación de ejecución local. Los nombres de variables se deben tomar de los archivos .env.example del commit que se instale.

8. Preparación del ambiente

La instalación documentada se realiza desde la raíz de PT_HS en Windows PowerShell con Docker Desktop iniciado. El equipo debe tener acceso a los archivos del repositorio y permiso para publicar los puertos locales. Para firmware se necesita además PlatformIO y una placa ESP32; para BLE móvil se necesita un development build.

Requisito	Comprobación / uso	No hacer
Docker Desktop	Debe estar iniciado; se verifica con docker version.	No levantar contenedores con variables secretas incompletas.
Compose	docker compose --env-file .env config --quiet.	No ignorar errores de interpolación.
Archivo .env	Se crea desde .env.example y conserva credenciales locales.	No subir .env ni copiar secretos en tickets.
ESP32/hardware	Necesario solo para instalación y prueba física IoT.	No conectar señales sin revisar compatibilidad eléctrica.
Development build móvil	Necesario para react-native-ble-plx.	No usar Expo Go como sustituto de BLE nativo.

9. Variables de entorno sin valores

Los nombres siguientes aparecen en docker-compose.yml o en la documentación del repositorio. Deben tomar sus valores del .env.example y del ambiente autorizado; este manual no fija contraseñas ni direcciones productivas.

Grupo	Variables documentadas	Uso
PostgreSQL	POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_PORT	Base, rol administrador y puerto publicado.
Aplicación BD	DB_USER, DB_PASSWORD, DB_INGEST_USER, DB_INGEST_PASSWORD	Roles hidro_smart_app e hidro_smart_ingest.
JWT	JWT_SECRET, JWT_EXPIRES_IN, JWT_REFRESH_EXPIRES_IN, JWT_ISSUER, JWT_AUDIENCE	Sesiones y validación de tokens.
Backend	BACKEND_PORT, FRONTEND_URL, CORS_ORIGIN, APP_TIMEZONE	API, origen permitido y zona horaria.
MQTT	MQTT_BROKER_URL, MQTT_PORT, MQTT_DEVICE_HOST, MQTT_DEVICE_PORT, MQTT_USERNAME, MQTT_PASSWORD	Broker del backend y host accesible por ESP32.
Correo	SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASSWORD, SMTP_FROM	Recuperación/notificaciones; credenciales reales no verificadas.
Frontend	VITE_API_URL, VITE_MQTT_DEVICE_HOST, VITE_MQTT_DEVICE_PORT	API y broker que el móvil/ESP32 puede alcanzar.

10. Instalación desde cero con Docker

Ejecute desde PT_HS y deténgase si un comando devuelve error. El orden respeta dependencias de salud, bootstrap de roles y migraciones.

13. Crear el archivo local: Copy-Item .env.example .env.

14. Revisar valores sin publicarlos y validar Docker: docker version.
15. Validar interpolación: docker compose --env-file .env config --quiet.
16. Levantar persistencia y broker: docker compose --env-file .env up -d postgres db-bootstrap mosquitto.
17. Validar migraciones: docker compose --env-file .env --profile tooling run --rm liquibase validate.
18. Consultar migraciones: docker compose --env-file .env --profile tooling run --rm liquibase status --verbose.
19. Aplicar migraciones: docker compose --env-file .env --profile tooling run --rm liquibase update.
20. Construir y levantar aplicación: docker compose --env-file .env up -d --build backend frontend.
21. Consultar servicios: docker compose --env-file .env ps.
22. Probar backend: Invoke-WebRequest http://localhost:3000/health.
23. Probar frontend: Invoke-WebRequest http://localhost:5173/health.

10.1 Arranque normal y actualización

Cuando la base ya está aplicada, el arranque normal documentado es docker compose --env-file .env up -d. Si se incorporan changesets nuevos, se ejecutan nuevamente validate, status --verbose y update antes de reiniciar o probar el backend.

10.2 Reinicio limpio: solo desarrollo

docker compose --env-file .env down --volumes --remove-orphans elimina el volumen local de PostgreSQL y datos de desarrollo. No es una actualización normal. Debe ejecutarse solo después de confirmar que el objetivo es reconstruir desde cero y que no contiene datos que deban conservarse.

11. Validación posterior a la instalación

Nivel	Prueba	Resultado que se busca
Servicios	docker compose ps y logs del servicio con error.	Postgres, db-bootstrap, Mosquitto, backend y frontend saludables.
BD	validate, status --verbose y consulta controlada.	Liquibase actualizado; roles y estructura disponibles.
API	GET /health, /health/live y /health/ready.	Proceso, BD y MQTT reportan estado esperado.
Frontend	GET /health y navegación inicial.	Nginx entrega la aplicación y /api/ se reenvía.
MQTT	Test-NetConnection localhost -Port 1883.	Broker escucha localmente; la telemetría se prueba después con payload controlado.
Integración	Backend npm test/check/test:integration según variables disponibles.	Resultados registrados; las pruebas omitidas se explican.

12. Instalación y aprovisionamiento del ESP32

Abra firmware en PlatformIO, seleccione el entorno esp32dev y mantenga el monitor a 115200. El firmware declara PubSubClient, ArduinoJson y NimBLE-Arduino. El flujo BLE guarda SSID, contraseña, host, puerto y deviceCode en NVS; el usuario final no debe escribir la IP del PC en una pantalla de la aplicación.

24. Energizar el ESP32 y confirmar que el hardware BLE sea visible.
25. Usar el servicio BLE documentado y escribir el JSON de configuración en la característica correspondiente.
26. Enviar mqttHost y mqttPort cuando el broker no sea el valor previamente guardado.
27. Leer estados wifi_connecting, wifi_failed, wifi_connected, mqtt_unreachable o provisioning_complete.

28. Confirmar que el ESP32 publique status y telemetry con el deviceCode esperado.
29. No registrar la contraseña Wi-Fi; el firmware documenta que no se publica en MQTT, PostgreSQL, Git ni logs.

13. Actualización, rollback y respaldo

- Registrar commit, fecha, ambiente, variables no secretas y estado de Liquibase antes de actualizar.
- Aplicar cambios de BD mediante Liquibase; no editar tablas manualmente para compensar diagramas.
- Conservar logs de build, migración y pruebas.
- Antes de reiniciar o eliminar volúmenes, confirmar backup y restauración; el flujo completo de backup/restore no está verificado en el corte.
- Si falla una verificación, detener la liberación y conservar la versión previamente validada según la política del ambiente.

14. Resolución de problemas de instalación

Problema	Revisión
config --quiet falla	Variables obligatorias, especialmente contraseñas y MQTT_DEVICE_HOST.
db-bootstrap no termina	PostgreSQL healthy, POSTGRES_PASSWORD, DB_PASSWORD y DB_INGEST_PASSWORD.
Liquibase no conecta	Servicio postgres:5432 dentro de Docker; no usar localhost:5433 desde el contenedor.
Backend unhealthy	Logs, conexión a BD, JWT_SECRET, usuario ingest y broker mqtt://mosquitto:1883.
Frontend no construye	VITE_API_URL, VITE_MQTT_DEVICE_HOST y VITE_MQTT_DEVICE_PORT.
ESP32 no publica	Host alcanzable desde la red del ESP32; localhost solo sirve para el equipo local.

Control de evidencia. Fuentes: PT_HS/README.md, docker-compose.yml, docs/estado-verificado-2026-09-26.md, BD_HS/docs/README.md, firmware/README.md, platformio.ini y package.json. Los pasos destructivos se describen para advertir, no para ejecutarlos automáticamente.